

Aura: Mental Clarity & Thought Vault

Document Type: Software Requirements & Architecture (SRS)

Platform: React Native (Expo) / Node.js / PostgreSQL

Version: 1.0.0

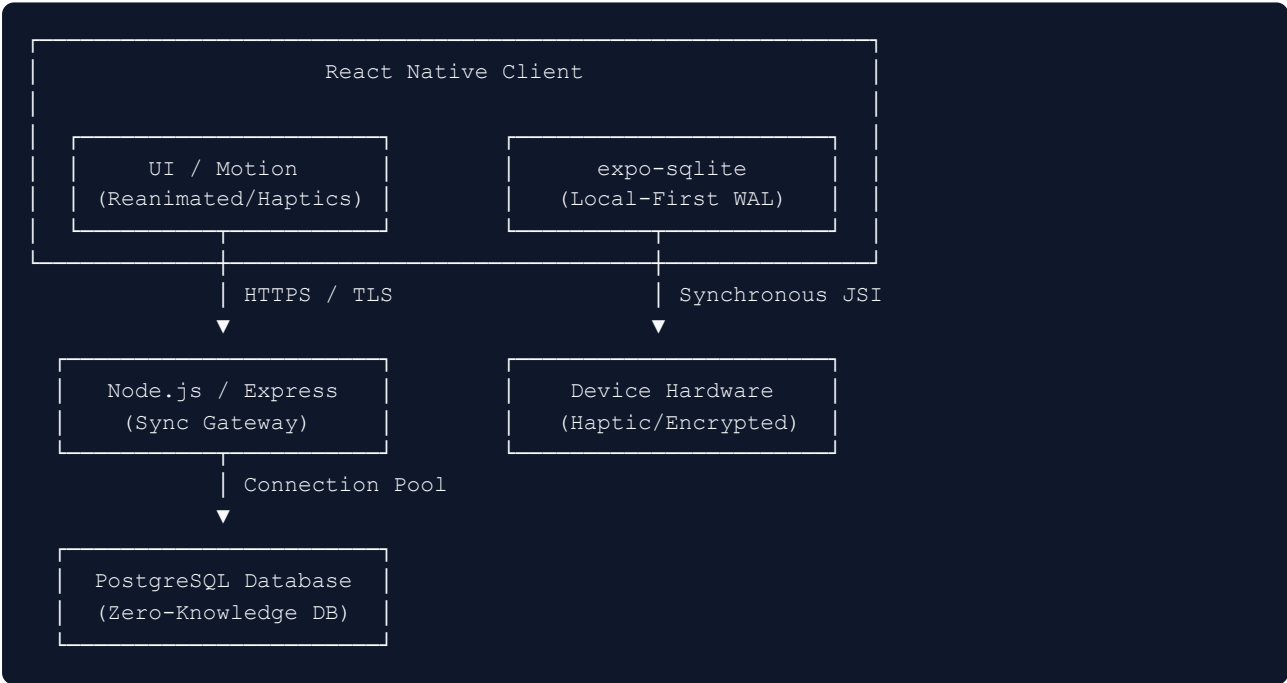
Date: September 2026

1. Executive Summary & Product Purpose

Aura is an offline-first mobile application designed to reduce acute cognitive friction, nighttime rumination, and anxiety loops. Traditional digital journals rely on persistent retention, forcing users to repeatedly re-read anxious entries. Aura replaces passive retention with psychological containment:

- **Thought Purge (Catharsis):** An ephemeral workspace where inputs are dissolved with real-time particle dissipation and tactile haptic feedback, leaving zero disk or memory footprints.
- **Time-Lock Vault (Compartmentalization):** Thoughts requiring attention later are encrypted and locked into local storage until a designated morning release window.

2. System Architecture



3. Database Specifications

3.1 Embedded Client Database (SQLite)

Runs directly on device hardware via `expo-sqlite` in Write-Ahead Logging mode for sub-millisecond I/O.

```
PRAGMA journal_mode = WAL;

CREATE TABLE IF NOT EXISTS vault_items (
  id TEXT PRIMARY KEY NOT NULL,
  type TEXT NOT NULL CHECK(type IN ('purged', 'time_locked')),
  content TEXT NOT NULL,
  locked_until INTEGER, -- Unix timestamp in milliseconds
  created_at INTEGER NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_locked_until ON vault_items (locked_until);
```

3.2 Central PostgreSQL Schema (Cloud Backup)

Encrypted zero-knowledge persistence tier.

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  device_id VARCHAR(128) UNIQUE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE vault_items (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  type VARCHAR(20) NOT NULL CHECK (type IN ('purged', 'time_locked')),
  encrypted_payload TEXT NOT NULL,
  locked_until TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_vault_user_lock ON vault_items(user_id, locked_until);
```

4. API Interface Contracts

Method & Route	Parameters / Body	Response	Description
<code>POST /api/vault/lock</code>	<code>{ deviceId, encryptedContent, lockDurationHours }</code>	<code>201 Created</code> <code>{ success: true, item: { id, locked_until } }</code>	Syncs encrypted payload to cloud vault with release timestamp.
<code>GET /api/vault/unlocked/:deviceId</code>	<code>deviceId</code> (path string)	<code>200 OK</code> <code>{ items: [...] }</code>	Retrieves records whose <code>locked_until</code> epoch has passed.

5. Design Tokens & Somatosensory Feedback Matrix

Token Name	Hex / Style Code	WCAG / Interaction Target
Canvas Background	#090A0F	Deep obsidian matte; zero OLED pixel burn.
Card Surface	#13161F	Slate surface with 1px border opacity.
Purge Accent	#38BDF8	Sky Blue; signals release and non-persistence.
Time-Lock Accent	#818CF8	Soft Indigo; signals compartmentalization.
Primary Typography	#F8FAFC	WCAG AAA compliance against dark background.
Haptic: Mode Switch	SelectionAsync()	Subtle tactile click indicating state shift.
Haptic: Purge Dissolve	ImpactFeedback.Medium	Cascading pulses ending in success feedback.
Haptic: Time Lock	ImpactFeedback.Heavy	Solid impulse confirming secure lock.

6. Implementation & Runbook

6.1 Mobile Application Setup

```
# Navigate into client directory
cd mobile

# Install core runtime modules
npm install expo-sqlite expo-haptics react-native-reanimated lucide-react-native

# Start local bundler
npx expo start -c
```

6.2 Sync Server Execution

```
# Navigate into backend directory
cd backend

# Install production dependencies
npm install express pg dotenv

# Initialize PostgreSQL database schema
psql -U postgres -d mental_clarity -f schema.sql

# Start server daemon
node server.js
```